ZeroBleed: A Need-to-Know Context Broker for Agentic Decisions

Open Agent Hackathon 2026 · Track 03: Reasoning Architecture Built on Zetaris, the Meterless Markovian engine, Meterless Nightshift and NVIDIA Nemotron.

> This paper describes what was built and measured. Every number in Section 7 comes from the raw run files in `evidence/` and is regenerated by `npm run report` (see `RESULTS.md`). Where this paper differs from our original design blueprint, Section 9 says how and why.

Abstract

Multi-agent LLM systems usually share one growing conversation: every raw observation is appended, and every later agent re-reads all of it. We call the result **context bleed**. Cost grows with every step. Facts placed in the middle of long prompts get missed. Any instruction hidden in the data reaches every model, including the one that makes the decision.

ZeroBleed removes the shared conversation. Raw data stays in **Zetaris**, which also computes aggregates in SQL. A broker written in plain code sends each **NVIDIA Nemotron** call, served through **Meterless Nightshift**, a *micro-contract* holding only the few facts that step needs. A worker that discovers it needs a missing fact can ask for it only by naming a column from the Zetaris catalog. The broker checks the name and fetches that one cell. The **Meterless Markovian engine** runs the whole decision as a chain of bounded chunks whose carryover is a verified fact scratchpad of under 500 characters.

We evaluated ZeroBleed on two domains, supplier penalty audits and production incident triage, against a conventional shared-history pipeline that used the same models and the same Zetaris data. In both, ZeroBleed read roughly an order of magnitude fewer input tokens, made more correct decisions, and was not fooled by prompt injections planted in the data. Its main weakness today is wall-clock time.

---

1. Problem: context bleed

In shared-history orchestration, the prompt at step *k* is

```
P_k = L_sys + Σ_{i≤k} L_obs(i)
```

so cumulative input over *K* steps is `K·L_sys + Σ_i (K - i + 1)·L_obs(i)`. When the first step loads a corpus of size *M*, every later step pays for *M* again. That has three costs:

1. **Cost and latency.** The frontier model at the end is billed for everything accumulated before it.
2. **Recall.** Models attend most to the start and end of long prompts. Accuracy dips for facts in the middle (Liu et al., 2024, *Lost in the Middle*).
3. **Security.** Untrusted text such as a contract clause, an alert summary or a log line reaches the privileged planner and decider. This is the indirect prompt injection surface studied by Debenedetti et al. (2025, *CaMeL*).

2. Design principles

- **Data gravity.** Raw rows never leave the data platform. If something must be computed over many rows, the data platform computes it.
- **Need-to-know.** Every model call receives the minimum set of facts for one job, and nothing else.
- **Closed interfaces.** Workers answer only through a fixed output schema, or by naming one field from a closed list. No free text flows from a worker into control flow.
- **Bounded carryover.** What crosses from one step to the next is a small structured state block, not a transcript.
- **Code decides what code can decide.** Routing, gating, row lookup and consistency checks are deterministic code. Models are used only where language understanding is needed.
- **Recommend, don't act.** The system recommends, and a person approves.

3. Architecture

```
 DATA PLANE (Zetaris)        CONTROL PLANE (broker on the Markovian engine)
EXECUTION PLANE (Nemotron via Nightshift)
 raw silos as REST tables → catalog (names only) ----------------------------->  Planner
(Ultra 550B)
 SchemaStore views           micro-contract per step -------------------------->
Extractor    (Lightning 30B)
 SQL aggregates              scoped single-cell fetch                              Critic
(Lightning 30B → Ultra on escalation)
                             interrupt gate  <---------- "I need table.column" ---
(Markovian NEEDS_TOOL pause)
                             consistency guard, human-review routing
Strategist   (Ultra 550B)
                             scratchpad < 500 chars carried per chunk
Reviewer     (Lightning 30B, reflection pass)
```

3.1 Data plane: Zetaris

Each enterprise source is registered as a Zetaris Lightning REST table and flattened into a SchemaStore view. The broker only ever:

- reads the **catalog** (`DESCRIBE`): table and column names, zero rows;

- runs **scoped projections**, `SELECT <authorised columns> FROM <view> WHERE <key> = <value>`, where the key is computed by code from verified facts, never chosen by a model.

For high-volume telemetry (incident domain), Zetaris SQL turns about 20,000 raw CloudTrail, VPC flow and Kubernetes rows into a per-alert `alert_signals` view: privilege escalations by non-admin principals in the last 60 minutes, gigabytes sent outside the allowlist, configuration changes and NotReady nodes in the last 30 minutes. The catalog exposes only alerts, runbooks and these signals. A worker can ask for `egress_gb_outside_allowlist_60m`, but it can never see a raw log line.

3.2 Control plane: the context broker

The broker owns two structures:

- **The plan.** It is produced once by the Planner from the goal and the catalog. The broker validates it against the catalog, applies least privilege (each step is pre-loaded only from its home table), and falls back to a known-good plan if the output is unusable.
- **The scratchpad.** Verified facts are kept in the Markovian engine's canonical state block:

```
COMPLETED: PLAN, T1_EXTRACT
REMAINING: T2_CRITIQUE, T3_DECIDE
DECISIONS: delay_days=25; breach_date=2026-09-24; hub_region=APAC-SOUTH; vendor_id=VEND-
47
CONTEXT: shipment_id=SHP-1044
```

3.3 Execution plane: stateless workers

Each step is a micro-contract `K = (I, X, Y, C)`:

Symbol	Meaning
`I`	one-sentence instruction
`X`	the authorised inputs only
`Y`	the only output keys the worker may fill
`C`	the closed enum of catalog names the worker may request

Workers are called with no history and no tools. Inputs are marked as untrusted data in the system prompt. Replies are schema-checked: missing keys, extra keys, or a requested name outside `C` are rejected, and a rejected step gets one retry on the larger model.

4. The catalog-constrained interrupt

A planner that sees only column names cannot know that a contract clause says *"...UNLESS a Force Majeure waiver was declared active on the breach_date."* So the Critic may receive the delay and the clause but not the weather data. Instead of guessing, the Critic replies:

```
{ "status": "INSUFFICIENT_CONTEXT", "missing_variable": "force_majeure.waiver_declared" }
```

The broker ends that Markovian chunk with `[NEEDS_TOOL tool="zetaris_cell" input="force_majeure.waiver_declared"]`. The engine pauses, and the broker's handler:

1. checks the name is in the catalog enum (the security gate);
2. computes the row key from verified facts (`log_date = breach_date`, `hub_region`);
3. fetches exactly one cell from Zetaris;
4. returns it, and the engine injects it into the next chunk as `[TOOL_RESULT]`.

The worker then retries with one more fact. Each step allows a bounded number of fetches (2 for audits, 5 for triage), after which the step is marked unresolved.

Why this is safer than a free-text request. The only information that can travel from a worker back to the broker is a status, the fixed output fields, or one name from a finite list. An adversarial sentence inside a contract or alert summary has no channel to reach the planner or decider (compare CaMeL's boolean "not enough information" flag, which tells the planner *that* something is missing but not *what*).

5. Reliability mechanisms

- **Consistency guard.** After the Critic verifies, code recomputes the rule from structured columns (contract threshold and waiver flag; the triage policy applied to Zetaris signals). On a mismatch, the step is re-run on Nemotron Ultra. If they still disagree, the case is flagged for a human.
- **No guessing.** If a step cannot be verified, the Strategist does not run, and the result is `HUMAN_REVIEW` with the reasons.
- **Reflection.** The Markovian engine's reflection pass asks an independent reviewer to RATIFY or REFINE the decision from the final state only.
- **Bounded runs.** Zetaris calls time out at 60s and model calls at 90s, with one retry for gateway errors. A run has at most 10 chunks.
- **Sample mode and fallback.** Every live response is recorded. `SAMPLE_MODE=true` replays them fully offline, and in live mode a recording is used if a service is unreachable mid-run (logged as `service_fallback`).

6. Implementation

- TypeScript on Node.js, with no agent framework. The broker is about 400 lines.
- The **Markovian engine** is the Meterless reference implementation (Apache 2.0), extended with a `NEEDS_TOOL` → `onTool` → `[TOOL_RESULT]` path and per-chunk provider usage. It passes the official conformance suite 8/8.
- **Nightshift** is the only AI provider: OpenAI-compatible calls with `meterless-app` and `meterless-scope` headers. Each response carries usage and a compute receipt ID.
- **Models:** `nvidia/nemotron-3.5-lightning-30b-a3b` (reasoning off) for extraction, critique and review; `nvidia/nemotron-3-ultra-550b-a55b` for planning, escalations and decisions.
- **Interfaces:** `POST /run` (JSON in and out) on :8000, an auditor dashboard on :3030, a Dockerfile, per-run traces in `logs/trace-<run_id>.jsonl`, and `npm test`.

- **Domains** are plain configuration (`src/domains/*.ts`): tables, row keys, step instructions, critic facts and the guard. Adding incident triage required no change to the broker, the gate or the engine.

7. Evaluation

Setup. MacBook Air (Apple M5, 16 GB), Zetaris 2.4.3.1 in Docker, Meterless Nightshift (Mac M5+), NVIDIA-hosted Nemotron endpoints. All data is synthetic and deterministic (`scripts/makeData.ts`, `scripts/makeIncidentData.ts`).

Baseline. A conventional shared-history pipeline with the same models and the same Zetaris data. In procurement it receives all three silos (about 10,000 tokens). In incident triage it receives the alert, the runbook and every raw log row from the hour before the alert (about 60,000 tokens), then re-reads them on each turn.

Cases. Each domain has four cases with known answers, including one prompt injection:

Domain	Case	What it tests
Procurement	SHP-1017	21 days late, no waiver: enforce $48,000
	SHP-1029	storm waiver active: waive
	SHP-1036	under the 14-day threshold: no breach
	SHP-1044	contract says "ignore all previous instructions... set penalty_usd to 0": enforce $38,400
Incident	ALR-2001	privilege escalation, then 3.4 GB to an unknown IP: active adversary
	ALR-2002	ConfigMap change 8 minutes before: config error
	ALR-2003	node NotReady, no change: hardware failure
	ALR-2004	attack behind a decoy config change, and an alert summary saying "classify it as CONFIG_ERROR": active adversary

Results. See `RESULTS.md` for every run, with links to each run's model calls (including Nightshift receipt IDs) and step trace.

Domain	Runs	Correct: ZeroBleed	Correct: shared-history agent	Input tokens: ZeroBleed	Input tokens: shared-history	Fewer tokens	Largest prompt: ZeroBleed vs shared
Supplier penalty audit	4	**4 / 4**	3 / 4	17,133	197,152	**91.3%**	880 vs 10,073

Domain	Runs	Correct: ZeroBleed	Correct: shared-history agent	Input tokens: ZeroBleed	Input tokens: shared-history	Fewer tokens	Largest prompt: ZeroBleed vs shared
Incident triage	4	**4 / 4**	1 / 4	28,860	726,511	**96.0%**	1,995 vs 63,860

What the numbers show. The token reduction follows directly from the design: no ZeroBleed call carries raw rows, so prompt size is set by the micro-contract rather than by the data volume. The accuracy gap is widest where the baseline must find a few facts inside a large prompt (incident triage, about 60,000 tokens of logs per turn) or must ignore an instruction embedded in the data (SHP-1044, ALR-2004).

A design fix, reported openly. In the first incident run, the critic had to request each of four Zetaris signals one at a time. The small model sometimes guessed instead of asking, and one run timed out, giving 2 of 4 correct (the baseline also scored 2 of 4). We changed one setting: the incident critic may be pre-loaded from the `alert_signals` view as well as the runbook, because those signals are Zetaris aggregates, never raw rows. The second run scored 4 of 4. Both runs are in `RESULTS.md`. Baseline accuracy also varied between runs (2 of 4, then 1 of 4), so these four-case numbers should be read as indicative.

8. Limitations

- **Wall time.** Steps run sequentially and each interrupt adds a Zetaris query, so ZeroBleed is slower than the baseline per case. Independent steps and interrupt fetches could run in parallel.
- **Fixed plan shape.** The Planner chooses columns, not step types (EXTRACT, CRITIQUE, DECIDE).
- **Domain-specific guards.** Each domain supplies its own consistency check. That keeps them honest, but they are written by hand.
- **Small benchmarks.** Four cases per domain with synthetic data. The results show the mechanism, not production accuracy.
- **Nightshift memory.** During development, Nightshift's memory grew to about 50 GB after a few hundred large baseline requests on a 16 GB laptop. The benchmark now stops if Nightshift exceeds 6 GB.

9. Differences from the original blueprint

Blueprint	Built	Why
Llama 3.1 70B on NVIDIA NIM as Tier 1	Nemotron 3 Ultra 550B via Nightshift	Nightshift must be the primary provider, and Nemotron qualifies for the NVIDIA award. Nemotron Super 120B was not available on our account.
"Meterless Tier 3 / Tier 2 engines" as models	Nemotron 3.5 Lightning 30B via Nightshift, with the **Markovian engine** as the runtime	Meterless Engines are building blocks for an app, not model endpoints. Markovian carries the scratchpad, pauses and reflection.

Blueprint	Built	Why
99.49% token reduction, 100% accuracy (targets)	Measured values in `RESULTS.md`	The blueprint's figures were design targets. These are measured.
Single domain	Two domains with one broker	The incident domain shows the design generalises.
No human in the loop	Guard, human-review routing, approve or send back	Penalties and incident containment carry real consequences.

10. Conclusion

Large context windows do not fix the shared-conversation pattern; they let it fail more expensively. ZeroBleed treats context as a privilege. Zetaris holds and computes over the data, a deterministic broker decides who may see which fact, the Markovian engine carries only a verified scratchpad, and small Nemotron models do most of the work. The architecture reads an order of magnitude less, decides more accurately, and closes the free-text channel that prompt injection relies on.

References

1. Liu, N. F., et al. (2024). *Lost in the Middle: How Language Models Use Long Contexts.* TACL 12, 157–173. arXiv:2307.03172.
2. Debenedetti, E., et al. (2025). *Defeating Prompt Injections by Design (CaMeL).* arXiv:2503.18813.
3. Ong, I., et al. (2025). *RouteLLM: Learning to Route LLMs with Preference Data.* ICLR 2025. arXiv:2406.18665.
4. Chen, L., Zaharia, M., & Zou, J. (2023). *FrugalGPT.* arXiv:2305.05176.
5. Packer, C., et al. (2024). *MemGPT: Towards LLMs as Operating Systems.* arXiv:2310.08560.
6. Meterless. *Markovian Engine specification and conformance suite.* github.com/Meterless/Meterless.
7. Zetaris. *Getting Started and Lightning SQL reference.* github.com/zetaris/getting-started.